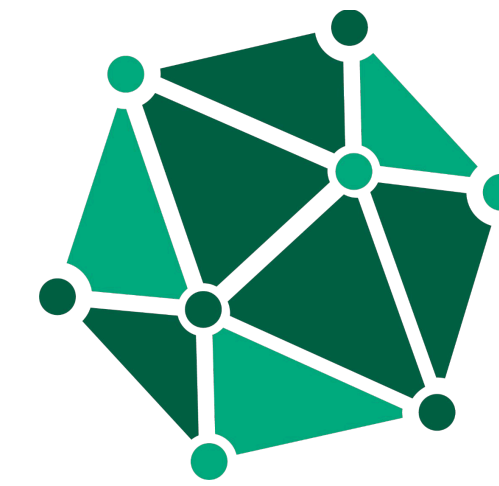


PRODUCT, UX AND ENGINEERING DOCUMENTATION

PROJECT OVERVIEW

Kevin Von Dumke - Full Stack Developer



NETLINK

Netlink is a modern communication platform designed to streamline how users connect, share, and collaborate across devices.

Start Here

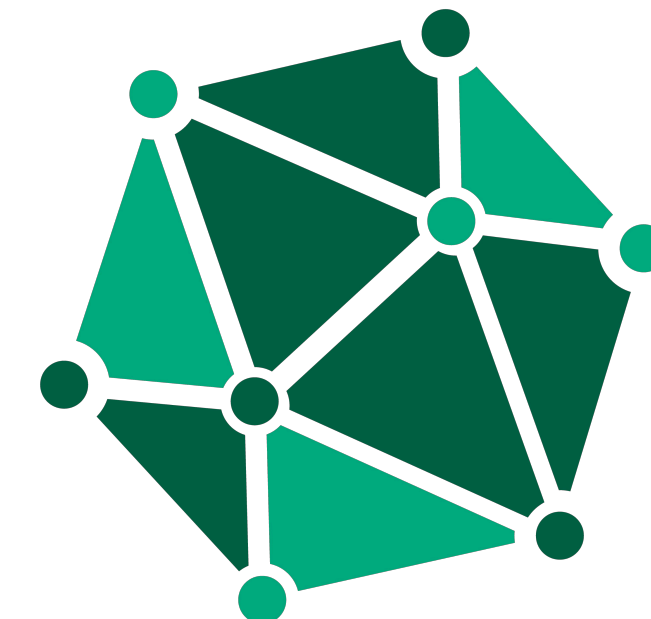


01

PROJECT OVERVIEW

SUMMARY

- 01 Introduction
- 02 Problem and Solution
- 03 Product Vision
- 04 Product Goals
- 05 Technology Stack
- 06 Core Product Features
- 07 Engineering Capabilities
- 08 Project Statistics
- 09 Challenges Solved
- 10 Scope and Limitations



INTRODUCTION

Netlink is a social platform built for network engineers and technical professionals.

It combines content sharing, relationship-building, messaging, and privacy controls in one workspace designed around professional networking rather than generic social media.

It solves the problem of fragmented technical networking by giving users a single place to publish updates, discover peers, follow relevant people, exchange messages, and manage visibility and safety controls.

It is for engineers, technical communities, and professionals who want a focused network for sharing expertise and building connections.

In short, Netlink is a professional social platform for people who work in networking and adjacent technical fields, designed to make knowledge-sharing and relationship management feel structured, useful, and secure.



PROBLEM

Technical professionals often rely on multiple disconnected tools to do things that should live in one place:

- Sharing updates.
- Finding peers.
- Managing relationships.
- Messaging.
- Protecting privacy.

Generic social platforms are noisy, while professional tools are often too rigid or too narrow.

That creates a friction for both communication and community-building.

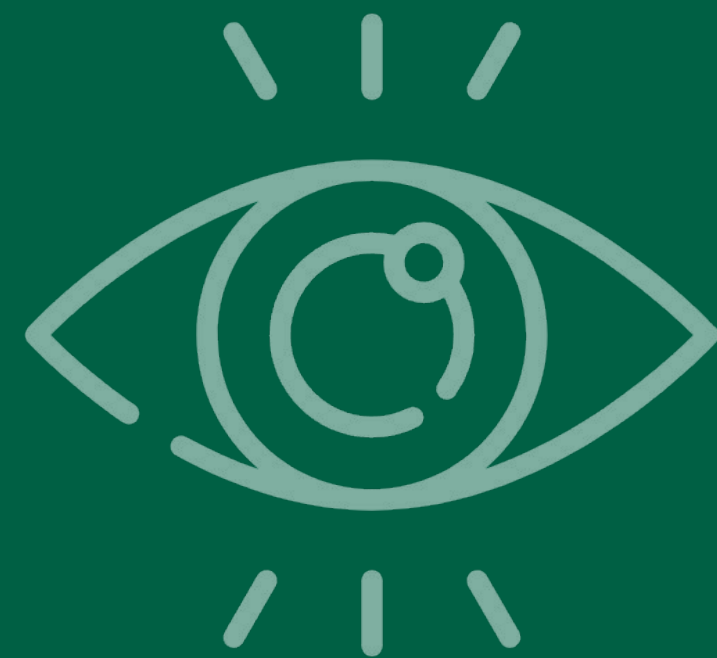
SOLUTION

Netlink addresses that gap with a purpose-built social experience for technical users.

It combines into one product:

- Posts.
- Feeds.
- Discovery.
- Followers.
- Favorites.
- Chat.
- Notifications.
- Privacy features.

The result is a platform that supports both public professional presence and controlled interpersonal communication.



PRODUCT VISION

Netlink is trying to become the default professional social space for network engineers: a place where technical identity, community participation, and practical communication live together.

The vision is not just to host content, but to shape a trusted network where professionals can build visibility, exchange ideas, discover people, and manage their public or private boundaries with confidence.

In that sense, Netlink is aiming to be both a network and a working environment for technical relationships.

What makes the vision distinct is the balance between openness and control.

PRODUCT GOALS

User Goals

- **Help** users publish technical updates, questions, and posts quickly.
- **Make** it easy to discover relevant people, profiles, and content.
- **Support** meaningful connection management.
- **Give** users control over visibility, privacy and interaction safety.
- **Keep** the experience focused and efficient for professional use.

Engineering Goals

- **Maintain** a clean client-server split with a typed API boundary.
- **Keep** social interactions reliable through consistent data modeling and relational integrity.
- **Support** scalable feed and content retrieval patterns.
- **Reduce** complexity in UI state through reusable composables.
- **Preserve** a token-based design system so visual changes stay consistent.

UX Goals

- **Make** the platform feel fast and predictable.
- **Keep** navigation simple across feed, profile, create, explore, connections and messages.
- **Reduce** clutter so the interface supports professional use rather than entertainment-driven browsing.
- **Provide** visible feedback for loading, empty states, and errors.

TECHNOLOGY STACK

Frontend

- **Vue 3** is a good fit because the UI is component-driven and stateful, with many reusable screens and social interaction flows.
- **Vite** keeps development fast.
- **TypeScript** improves maintainability.
- **Pinia** handles shared application state cleanly.
- **Vue Router** supports route-based product structure.
- **Axios** provides a straightforward API client layer.

Backend

- **Node.js + Express** fit the product because the backend is API-centric and route-driven.
- **TypeScript** helps keep request and response contracts consistent across the app.

Deployment

- The frontend is deployed via **Netlify** and backend via **Render**.

Database

- **PostgreSQL** is a strong fit for a social platform because the data is relational and interconnected.

ORM

- **Prisma** is used because it gives the project a typed schema and a clearer model of complex relations.

Authentication

- **JWT** suits the app because the frontend and backend are separated and the platform needs stateless session handling for protected routes and authenticated API calls.

Design

- **CSS variables** and **CYDERX** token system. The design layer is built around reusable tokens, custom fonts, and style variables. That gives the UI a consistent visual language.

CORE PRODUCT FEATURES

Module	Features	Status
Authentication	Login, register, protected routes	✓
Home / Feed	Dashboard feed, feed navigation	✓
Post Management	Create, edit, delete, like, comment, repost	✓
Discovery	Explore users/content and search browsing	✓
Connections	Follow, unfollow, followers, following, favorites	✓
Messaging	Direct chats, messages, unread counts	✓
Profile & Safety	Profile pages, notifications, block/mute/report	✓

ENGINEERING CAPABILITIES

Capability	Status
TypeScript	✓
Vue 3 Composition API	✓
Vue Router + Auth Guards	✓
Pinia State Management	✓
Axios API Service Layer	✓
Express REST API Backend	✓
Prisma ORM Schema/Models	✓

PostgreSQL Data Layer	✓
JWT Authentication	✓
Middlewares Error Handling	✓
Cursor-based Pagination	✓
Shared composables / State Hooks	✓
Design Tokens via CSS variables	✓
Frontend Unit Test with Vitest	✓

IMPLEMENTED

Capability	Status
GraphQL API	☐
WebSocket RealTime Sync	☐
SSR / SSG	☐
PWA / Offline Mode	☐
Internationalization / Localization	☐
Storybook / Component Explorer	☐
End-to-End Test Suite	☐
CI / CD Pipeline Config	☐
Docker / Container Setup	☐
Redis / Cache Layer	☐
Search Indexing Engine	☐
Analytics / Telemetry	☐
RBAC / Admin Console	☐
Dark Mode / Theme Switching	☐

NOT IMPLEMENTED YET

Supporting evidence is in the frontend router, backend Prisma schema, and design token file, plus the frontend/backend package scripts: router/index.ts, schema.prisma, utilities.css, frontend/backend package.json

PROJECT STATISTICS

Description	Value	Basis
Screens	14	Route-backed views in the frontend router.
User Flows	8	Core product journeys inferred from the routed feature set.
UI Components	11	Reusable Vue components excluding route screens.
API Endpoints	54	Express route handlers across backend routers.
Database Tables	16	Prisma models in the schema.
Vue Components	27	All .vue files in the frontend app.
Composables	2	usePosts and useUserData.
Design Tokens	64	CSS custom properties in utilities.css.

CHALLENGES SOLVED

1. Keeping authenticated pages protected without duplicating logic.

Solution → Route guards enforce access control before users reach protected views.

2. Loading large social feeds without overfetching.

Solution → Cursor-based pagination keeps feed requests incremental and efficient.

3. Managing many social entities with consistent relationships.

Solution → Prisma models define and relations in one schema.

4. Supporting multiple interaction types on a single post.

Solution → The data model separates them while keeping them connected.

5. Handling privacy and safety behavior without breaking the social graph.

Solution → The backend supports block, mute, report, privacy mode and visibility rules.

6. Keeping shared UI state manageable across many screens.

Solution → Composables and shared action modules centralize post and user behavior.

SCOPE AND LIMITATIONS

Current Scope

Netlink currently focuses on a professional social platform for technical users.

The implemented scope includes authentication, routing, dashboards, posts, discovery, profiles, connections, messaging, notifications, privacy controls and a relational backend that supports those workflows.

Not Included

The current project does not have an implementation for

- GraphQL
- Realtime WebSockets
- PWA / Offline Support
- SSR / SSG
- I18N / Localization
- StoryBook
- Native Mobile App
- Analytics / Telemetry
- RBAC / Admin Tooling
- Docker / Container Setup
- CI / CD Pipeline